Champion-level drone racing using deep reinforcement learning

https://doi.org/10.1038/s41586-023-06419-4Received: January 5, 2023 Accepted: July 10, 2023 Online publication date: August 30, 2023 Open Access

Check for updates

Elia Kaufmann[1]✉, Leonard Bauersfeld[1], Antonio Loquercio[1], Matthias Müller[2],Vladlen Koltun[3] and Davide Scaramuzza[1]

First-person view (FPV) drone racing is a televised sport in which professional pilots operate high-speed craft through three-dimensional tracks. Each pilot observes the environment from the viewpoint of t heir own drone via a video stream transmitted from an onboard camera. Enabling autonomous drones to reach the level of professional pilots is extremely challenging, because the robot needs to fly at physical limits while estimating its own speed and position in the track using only onboard sensors[1]. Here, we introduce Swift, an autonomous system capable of racing physical craft at the level of a hu man world champion. The system combines deep reinforcement learning (RL) in simulation with data collected in the physical world. Swift competed head-to-head against three human champions in real-world races, including world champions from two international leagues.

Swift won several races against each human champion in multiple matches and demonstrated t he fastest recorded race time. This work represents a milestone in mobile robotics and machine intelligence[2], , and may inspire the deployment of hybrid learning-based solutions in other physical systems.

Deep reinforcement learning[3] has driven some of the latest advances in artificial intelligence. Policies trained via deep reinforcement learning have surpassed humans in complex competitive games, including Atari[4–6], Go[5, 7–9], chess[5,9], , StarCraft [10], , Dota 2 (reference11), and Gran Turismo[12,13]. These impressive demonstrations of machine intelligence capabilities are mainly confined to simulation and board game environments, which support policy search in exact replicas of the test conditions. Overcoming this limitation and demonstrating champion-level performance in physical competitions is a long-standing problem in autonomous mobile robotics and artificial intelligence[14–16].

FPV drone racing is a televised sport in which highly trained human pilots push their aircraft to physical limits during high-speed agile maneuvers (Fig.1a). The aircraft used in FPV racing are quadrotors, which are among the most agile machines ever built (Fig.1b). During races, these aircraft exert forces exceeding five times their own weight or more, reach speeds above 100 km h^{-1}, and achieve accelerations of several times gravity, even in confined spaces. Each aircraft is remotely controlled by a human pilot, who wears a head-mounted device displaying the video stream from an onboard camera, thereby creating an immersive "first-person view" experience (Fig.1c).

Attempts to create autonomous systems that reach human-pilot performance date back to the first autonomous drone racing competition in 2016 (Reference17). A series of innovations followed, including the use of deep networks to identify the position of the next gate[18–20], transferring racing policies from simulation to reality[21,22] , and accounting for uncertainty in perception[23,24]. The 2019 AlphaPilot autonomous drone racing competition showcased some of the best research in the field[25]. However, the time required by the top two teams to complete the course was still nearly twice that of professional human pilots[26,27]. More recently, autonomous systems have begun to reach expert-level human performance[28–30]. However, these works rely on near-perfect state estimation, which is provided by an external motion capture system. This makes the comparison with human pilots unfair, because humans only have access to onboard observations on the drone.

In this paper, we introduce Swift, an autonomous system that enables a quadrotor to race at the level of a human world champion using only onboard sensors and computation. Swift consists of two key modules: (1) a perception s ystem that converts high-dimensional visual and inertial information into a low-dimensional representation; and (2) a control policy that receives the low-dimensional representation produced by the perception system and generates control commands.

The control policy is represented by a feedforward neural network and trained in simulation using model-free on-policy deep reinforcement learning[31]. To bridge the differences in perception and dynamics between simulation and the physical world, we utilized non-parametric empirical noise models estimated from data collected on the physical system. These empirical noise models have been shown to be crucial for successfully transferring control policies from simulation to reality.

We evaluated Swift on a physical track designed by professional drone racing pilots (Figure1a). The track contains seven square gates, distributed within a volume of$30 \times 30 \times 8$ m, forming a lap with a length of 75 m. On this track, Swift raced against three human champions: Alex Vanover, 2019 Drone Racing League world champion; Thomas Bitmatta, two-time MultiGP International Open World Cup champion; and Marvin Schaepper, three-time Swiss national champion. The quadrotors used by Swift and the human pilots had the same weight, shape, and propulsion system. They are similar to the drones used in international competitions.

Human pilots practiced on the track for one week. After this week of practice, each pilot competed in several head-to-head races against Swift (Figure1a,b). In each head-to-head race, two drones

[1]Robotics and Perception Group, University of Zurich , Zurich, Switzerland.[2]Intel Labs, Munich, Germany.[3]Intel Labs, Jacks on , Wyoming, USA.✉Email:ekaufmann@ifi.uzh.ch



图 1| **无人机竞速**。**a**，Swift（蓝色）与Alex Vanover，2019年无人机竞速联盟世界冠军（红色），进行正面交锋。赛道包含七个方形门，每圈必须按顺序通过。要赢得比赛，竞争者必须比对手先完成连续三圈。**b**，Swift的近距离视图，用蓝色LED灯照亮，以及一架人类驾驶无人机，用红色LED灯照亮。本工作中使用的自主无人机依赖

Relying solely on onboard sensor measurements, without the support of external infrastructure (such as motion capture systems).**c**, From left to right: Thomas Bitmatta, Marvin Schaepper, and Alex Vanover pilot drones through the track. Each pilot wears a head-mounted device that displays in real time the video stream transmitted from the camera on their aircraft. The head-mounted device provides an immersive "first-person view" experience.**c**, Photograph by Regina Sablotny.

(one controlled by a human pilot, one controlled by Swift) start from the takeoff platform. The race is started by an audio signal. The first aircraft to complete three full laps of the track, passing through all gates in the correct order on each lap, wins the race.

Swift won multiple races against each human pilot and set the fastest race time recorded during the event. To our knowledge, our work is the first to achieve world-champion-level performance by an autonomous mobile robot i n real-world competitive sports.

The Swift System

Swift combines learning-based methods with traditional algorithms to map onboard sensor readings to control commands. This mapping consists of two parts: (1) an observation policy, which distills high-dimensional visual and inertial information into a task-specific low-dimensional encoding; and (2) a control policy, which converts the encoding into control commands for the drone. A schematic overview of the system is shown in Figure 2.

The observation policy includes a visual-inertial estimator[32,33], which operates in conjunction with a gate detector[26], ,

The gate detector is a convolutional neural network used to detect racing gates in onboard images. The detected racing gates are then used to estimate the drone's global position and orientation along the track. This is accomplished through a camera relocalization algorithm[34] combined with a track map. Then, a Kalman filter combines the global pose estimates obtained from the gate detector with the estimates from the visual-inertial estimator, resulting in a more accurate representation of the robot state. The control policy is represented by a two-layer perceptron that maps the output of the Kalman filter to control commands for the aerial vehicle. The policy is trained in simulation using on-policy model-free deep reinforcement learning[31]. During training, the policy maximizes a reward that combines progress toward the next racing gate[35] with a perception objective that rewards keeping the next gate within the camera's field of view. Seeing t he next gate is rewarded because it improves the accuracy of the pose estimation.

Without mitigating the gap between simulation and reality, optimizing the policy purely in simulation leads to poor performance on physical hardware. These discrepancies are mainly caused by two factors: (1) differences bet ween simulated dynamics and real dynamics; and (2) noisy estimates of the robot state by the observation policy when receiving real sensor data. We mitigate these discrepancies by collecting a small amount of data in the real world and using these data to improve the fidelity of the simulator.

Specifically, when a drone races on a track, we record the robot's onboard sensory observations, as well as high-precision pose estimates from a motion capture system. During this data collection phase, the robot is controlled by a policy trained in simulation and running based on the pose estimates provided by the motion capture system. The recorded data can identify characteristic failure modes of perception and dynamics observed on the track. The complexity of these perceptual failures and unmodeled dynamics depends on the environment, platform, track, and sensors. Perceptual residuals and dynamic residuals are modeled using Gaussian process[36] and *k*-nearest neighbor regression, respectively. The motivation for this choice is that we empirically

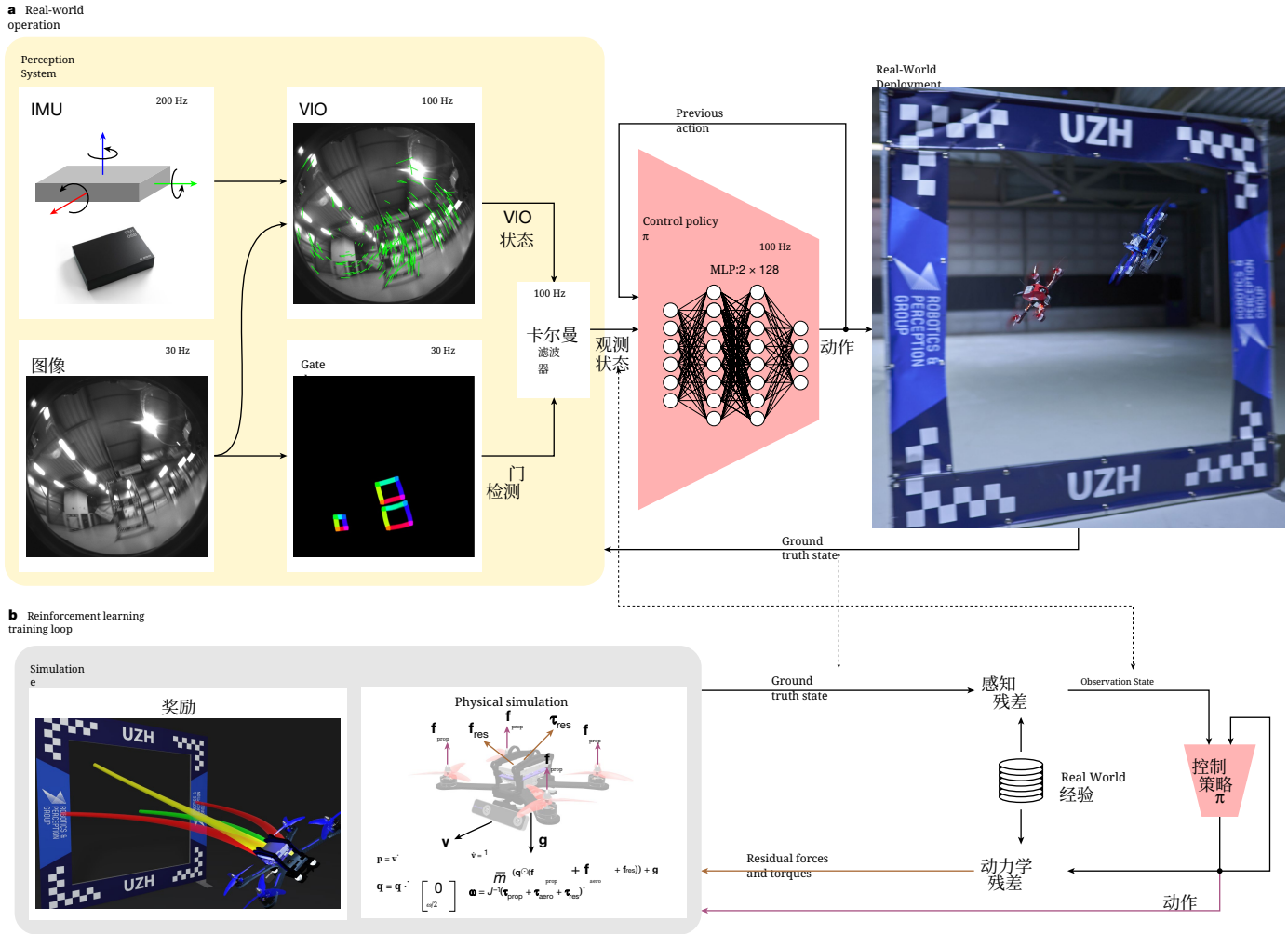


Figure 2 | Swift system. Swift consists of two key modules: a perception system that converts visual and inertial information into low-dimensional state observations; and a control policy that maps these state observations to control commands. The control commands specify the desired collective thrust and body angular rates, the same control modality used by human pilots.**a**, the perception system consists of a VIO module that computes metric estimates of t he drone's state from camera images and high-frequency measurements obtained from an inertial measurement unit (IMU). The VIO estimates are combined with a neural network t hat detects the corners of racing gates in images

流。角点检测被映射为三维位姿，并使用卡尔曼滤波器与VIO估计融合。**b**，我们使用无模型同策略深度强化学习在仿真中训练控制策略。训练期间，策略最大化一个奖励，该奖励将朝向下一个赛道门中心的进展与保持下一个门位于摄像头视场内的感知目标相结合。为了将竞速策略从仿真迁移到物理世界，我们用车辆感知和动力学的数据驱动残差模型增强仿真。这些残差模型是从赛道上收集的真实世界经验中辨识得到的。MLP，多层感知机。

find that perceptual residuals are stochastic, while dynamic residuals are largely deterministic (Extended Data Fig. 1). These residual models are integrated into the simulation, and the racing policy is fine-tuned in this augmented simulation. This method is related to the empirical actuator model used for sim-to-real transfer in reference37, but further incorporates empirical modeling of the perception system and also considers randomness in the platform's state estimation.

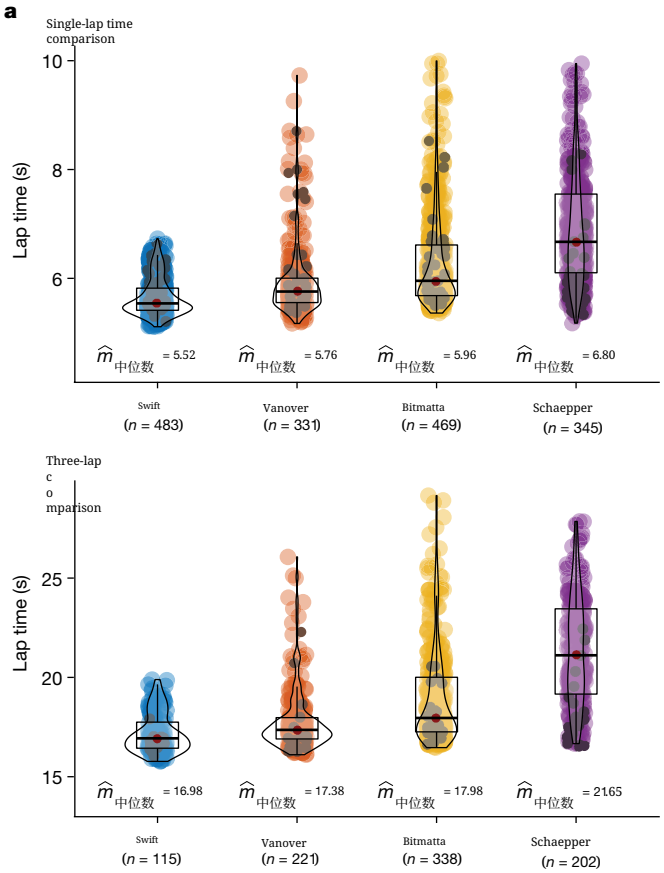
我们针对Swift的每个组件，在扩展数据中报告的受控实验里进行了消融。此外，我们与最近采用传统方法解决自主无人机竞速任务的工作进行了比较，包括轨迹规划和模型预测控制（MPC）。尽管此类方法在理想条件下（如简化动力学和完美已知机器人状态）能达到与我们的方法相当甚至更优的性能，但当其假设被违反时，性能便会崩溃。我们发现依赖预计算路径的方法[28,29] 对噪声感知和动力学特别敏感。没有任何传统方法

Even with high-precision state estimation from a motion capture system, it still failed to achieve competitive lap times comparable to those of Swift or human world champions. See the extended data for a detailed analysis.

结果

Drone racing is conducted on tracks designed by world-class external FPV pilots. The tracks feature typical and challenging maneuvers, such as the Split-S (Figure1a(top right) and Figure4d). Even if a crash occurs, as long as the aircraft can still fly, the pilot may continue the race. If both drones crash and are unable to complete the track, the drone that has traveled farther along the track wins.

As shown inFigure 3b, Swift won 5 out of 9 races against A. Vanover, 4 out of 7 races against T. Bitmatta, and 6 out of 9 races against M. Schaepper. Of the 10 l osses recorded by Swift, 40% were due to collisions with opponents, 40% were due to collisions with gates, and 20% were due to the drone being slower than the human pilot. Overall, Swift



b Head-to-head race results

	数量 Number of races	最佳完赛时间	胜场	负场	Win 比率
A. Vanover 对阵 Swift	9	17.956 s	4	5	0.44
T. Bitmatta 对阵 Swift	7	18.746 s	3	4	0.43
M. Schaepper vs. Swift	9	21.160 s	3	6	0.33
Swift vs. human pilot	25	17.465 s	15	10	0.60

Figure 3 | Results. a, Single-lap time results. We compare Swift with human pilots in time trials. Single-lap time denotes the best single-lap time and best average time achieved over rounds of three consecutive laps. The reported statistics are computed over a week-long dataset recorded on the track, corresponding to 483 (115) data points for Swift, 331 (221) for A. Vanover, 469 (338) for T. Bitmatta, and 345 (202) for M. Schaepper. The first number is the number of laps, and the second is the number of three-consecutive-lap rounds. The darker points in each distribution correspond to laps flown under race conditions. **b**, Head-to-head results. We report the number of head-to-head races flown by each pilot, the number of wins and losses, and the win rate.

Won the most races against each human pilot. Swift also achieved the fastest recorded race time, leading the best time set by a human pilot (A. Vanover) by half a second.

图4和扩展数据表1d分析了Swift和每位人类飞行员飞出的最快圈速。尽管Swift在全局上快于所有人类飞行员，但在赛道的各个单独赛段上并非都更快（扩展数据表1）。Swift在起步阶段以及诸如半滚倒转等急转弯中始终更快。起步时，Swift的反应时间更短，从出发台上起飞平均比人类飞行员早120 ms。此外，它加速更快，并在进入第一个门时达到更高的速度（扩展数据表1d，赛段1）。在急转弯中，如图4c,d所示，Swift找到了更紧凑的机动动作。一种假设是，Swift在比人类飞行员更长的时间尺度上优化轨迹。已知无模型强化学习可以通过价值函数优化长期奖励[38]。相反，人类飞行员在更短的时间尺度上规划其运动，最多提前一个门[39]。这一点很明显，例如在半滚倒转中（图4b,d），人类飞行员在机动动作开始时和结束时更快，但总体更慢（扩展数据表1d，赛段3）。此外，人类飞行员比Swift更早调整飞行器朝向以面向下一个门（图4c,d）。我们提出，人类飞行员习惯于将即将到来的门保持在视野中，而Swift已学会在依赖其他线索执行某些机动动作，例如惯性数据以及针对周围环境特征的视觉里程计。总体而言，在整个赛道上取平均，自主无人机实现了最高的平均速度，找到了最短的赛车线，并在整个比赛过程中设法将飞行器保持在更接近其驱动极限的状态，如平均推力和功率消耗所示（扩展数据表1d）。

We also compared the performance of Swift with human champions in time trials (Figure 3a). In time trials, a single pilot flies the track alone, and t he remaining number of laps is decided by the pilot. We aggregated ti me-trial data from the practice week and the competition, including training flights (Figure 3a, colored) as well as laps flown under race conditions (Figure 3a, black). For each participant, we used more than 300 laps to compute statistics. The autonomous drone more consistently pursued fast lap times, exhibiting a lower mean and variance. In contrast, human pilots decided lap by lap whether to accelerate, so lap times had a higher mean and variance in both training and competition. The ability to adjust flight strategy enabled human pilots to maintain a slower pace when they judged themselves to be clearly ahead, in order to reduce the risk of crashing. The autonomous drone is unaware of its opponents' situation and pursues the fastest expected completion time at all times, which may cause it to take too much risk when leading and too little risk when behind.[40].

讨论

FPV drone racing requires real-time decision-making based on noisy and incomplete sensory input from the physical environment. We present an autonomous physical system that achieves champion-level performance in t his sport, matching—and sometimes exceeding—the performance of human world champions. Our system has certain structural advantages over human pilots. First, it exploits inertial data from an onboard inertial measurement unit[32]. This is analogous to the human vestibular system[41], , which human pilots do not use because they are not physically located inside the vehicle and cannot feel the accelerations acting on it. Second, our system benefits from lower sensorimotor latency (40 ms for Swift, compared with an average of 220 ms for expert human pilots[39]). On the other hand, the limited refresh rate of t he camera used by Swift (30 Hz) can be regarded as a structural advantage for human pilots, whose cameras have a refresh rate four times faster (120 Hz), thereby improving their reaction time[42].

Human pilots possess impressive robustness: they are able to crash at full speed and—if the hardware still functions—continue flying and complete the course. Swift has not been trained to recover after a crash. Human pilots are also robust to changes in environmental conditions (e.g., lighting), which can significantly alter the appearance of the track. In contrast, Swift's perception system assumes that the environment's appearance matches what was observed during training. If this assumption does not hold, the system may fail. Training the gate detector and residual observation model under diverse conditions can provide robustness to appearance changes. Addressing these limitations would enable the proposed method to be applied in autonomous drone racing competitions, where access to the environment and the drone is restricted.[25].

Although limitations and unfinished work remain, achieving world-champion-level performance by an autonomous mobile robot in a popular physical sport is a milestone in robotics

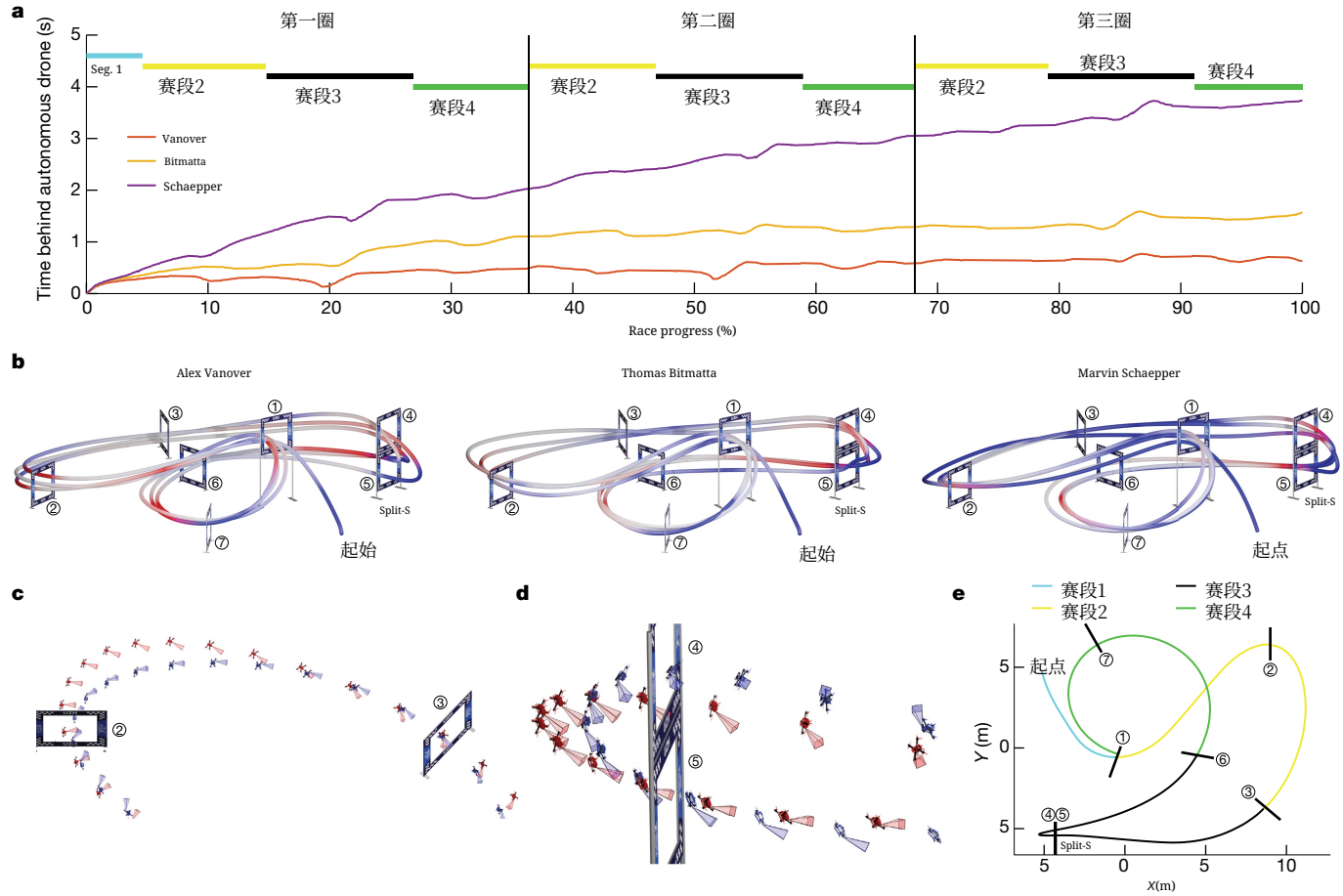


Figure 4 | analysis.a, comparison of each pilot's fastest race, expressed as the time behind Swift. The time difference with the autonomous drone is calculated based on the time after it passes the same position on the track. Although Swift is globally faster than all human pilots, it is not necessarily faster on each individual segment of the track.**b**, visualization of positions where human pilots are faster (red) and slower (blue) compared with the autonomous drone. Swift is consistently faster at the start and in sharp turns (such as the half roll inverted turn).**c**, analysis of the maneuver after gate 2. Swift is blue, Vanover is red. In this segment, Swift gains a time advantage over the human pilot by executing a tighter turn while maintaining comparable speed.**d**, analysis of the half roll inverted maneuver. Swift is blue, Vanover is red. The half roll inverted is the most challenging segment on the track, requiring carefully coordinated roll and pitch motions to form a descending half roll through two gates. In this segment, Swift gains a time advantage over the human pilot by executing a tighter turn with less overshoot.**e**, schematic diagram of the track segments used for analysis. Segment 1 is passed once at the start, while segments 2–4 are passed on each lap (three times in total during the race).

and machine intelligence. This work may inspire the deployment of hybrid learning-based solutions in other physical systems, such as autonomous ground vehicles, aircraft, and personal robots, across a wide range of domains.

Online content

All methods, additional references, Nature Portfolio reporting summary, source data, extended data, supplementary information, acknowledgements, peer review information; author contributions and conflict of interest details; and data and code availability statements are available athttps://doi.org/10.1038/s41586-023-06419-4.

1. De Wagter, C., Paredes-Vallés, F., Sheth, N. & de Croon, G. Rapid learning in autonomous drone racing.*Nat. Mach. Intell.* **3**, 923 (2021). 2. Hanover, D. et al. Autonomous drone racing: a survey. Preprint at https://arxiv.org/abs/2301.01755 (2023). 3. Sutton, R. S. & Barto, A. G. Reinforcement Learning: An Introduction (MIT Press, 2018). 4. Mnih, V. et al. Human-level control through deep reinforcement learning.*Nature* **518**, 529–533 (2015). 5. Schrittwieser, J. et al. Mastering Atari, Go, chess and shogi by planning with a learned model.*Nature* **588**, 604–609 (2020). 6. Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K. O. & Clune, J. First return, then explore.*Nature* **590**, 580–586 (2021).

7. Silver, D. et al. Mastering the game of Go with deep neural networks and tree search. Nature 529, 484–489 (2016). 8. Silver, D. et al. Mastering the game of Go without human knowledge. Nature 550, 354–359 (2017). 9. Silver, D. et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science 362, 1140–1144 (2018). 10. Vinyals, O. et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature 575, 350–354 (2019). 11. Berner, C. et al. Dota 2 with large scale deep reinforcement learning. Preprint at https://arxiv.org/abs/1912.06680 (2019). 12. Fuchs, F., Song, Y., Kaufmann, E., Scaramuzza, D. & Dürr, P. Super-human performance in Gran Turismo Sport using deep reinforcement learning. IEEE Robot. Autom. Lett. 6, 4257–4264 (2021). 13. Wurman, P. R. et al. Outracing champion Gran Turismo drivers with deep reinforcement learning. Nature 602, 223–228 (2022). 14. Funke, J. et al. in Proc. 2012 IEEE Intelligent Vehicles Symposium 541–547 (IEEE, 2012). 15. Spielberg, N. A., Brown, M., Kapania, N. R., Kegelman, J. C. & Gerdes, J. C. Neural network vehicle models for high-performance auto mated driving. Sci. Robot. 4, eaaw1975 (2019). 16. Won, D.-O., Müller, K.-R. & Lee, S.-W. An adaptive deep reinforcement learning framework enables curling robots with human-like performance in real-world conditions. Sci. Robot. 5, eabb9764 (2020). 17. Moon, H., Sun, Y., Baltes, J. & Kim, S. J. The IROS 2016 competitions. IEEE Robot. Autom. Mag. 24, 20–29 (2017). 18. Jung, S., Hwang, S., Shin, H. & Shim, D. H. Perception, guidance, and navigation for indoor autonomous drone racing using deep learning. IEEE Robot. Autom. Lett. 3, 2539–2544 (2018). 19. Kaufmann, E. et al. in Proc. 2nd Conference on Robot Learning (CoRL) 133–145 (PMLR, 2018). 20. Zhang, D. & Doyle, D. D. in Proc. 2020 IEEE Aerospace Conference, 1–11 (IEEE, 2020).

21. Loquercio, A. et al. Deep drone racing: from simulation to reality via domain randomization. IEEE Trans. Robot. 36, 1–14 (2019). 22. Loquercio, A. et al. Learning high-speed flight in the wild. Sci. Robot. 6, eabg5810 (2021). 23. Kaufmann, E. et al. in Proc. 2019 International Conference on Robotics and Automation (ICRA) 690–696 (IEEE, 2019). 24. Li, S., van der Horst, E., Duernay, P., De Wagter, C. & de Croon, G. C. Visual model-predictive localization for computationally efficient autonomous racing of a 72-g drone. J. Field Robot. 37, 667–692 (2020). 25. A.I. is driving drones (very, very slowly). https://www.nytimes.com/2019/03/26/technology/alphapilot-ai-drone-racing.html (2019). 26. Foehn, P. et al. AlphaPilot: autonomous drone racing. Auton. Robots 46, 307–320 (2021). 27. Wagter, C. D., Paredes-Vallé, F., Sheth, N. & de Croon, G. Perception, state estimation, and control behind the winning solution of the 2019 Artificial Intelligence Robotic Racing Competition. Field Robot. 2, 1263–1290 (2022). 28. Foehn, P., Romero, A. & Scaramuzza, D. Time-optimal planning for quadrotor waypoint flight. Sci. Robot. 6, eabh1221 (2021). 29. Romero, A., Sun, S., Foehn, P. & Scaramuzza, D. Model predictive contour control for time-optimal quadrotor flight. IEEE Trans. Robot. 38, 3340– 3356 (2022). 30. Sun, S., Romero, A., Foehn, P., Kaufmann, E. & Scaramuzza, D. A comparative study of nonlinear MPC and differential-flatness-based control for quadrotor agile flight. IEEE Trans. Robot. 38, 3357– 3373 (2021). 31. Schulman, J., Wolski, F., Dhariwal, P., Radford, A. & Klimov, O. Proximal policy optimization algorithms. Preprint at https://arxiv.org/abs/1707.06347 (2017). 32. Scaramuzza, D. & Zhang, Z. Encyclopedia of Robotics (eds Ang, M., Khatib, O. & Siciliano, B.) 1–9 (Springer, 2019). 33. Huang, G. in Proc. 2019 International Co nference on Robotics and Automation (ICRA) 9572–9582 (IEEE, 2019). 34. Collins, T. & Bartoli, A. Infinitesimal plane-based pose estimation. Int. J. Comput. Vis. 109, 252–286 (2014). 35. Song, Y., Steinweg, M., Kaufmann, E. & Scaramuzza, D. in Proc. 2021 IEEE/RSJ International Conference o n Intelligent Robots and Systems (IROS) 1205–1212 (IEEE, 2021).

36. Williams, C. K. & Rasmussen, C. E. Gaussian Processes for Machine Learning (MIT Press, 2006). 37. Hwangbo, J. et al. Learning agile and dynamic motor skills for legged robots. Sci. Robot. **4**, eaau5872 (2019). 38. Hung, C.-C. et al. Optimizing agent behavior over long time scales by transporting value. Nat. Commun. **10**, 5223 (2019). 39. Pfeiffer, C. & Scaramuzza, D. Human-piloted drone racing: Visual processing and control. IEEE Robot. Autom. Lett. **6**, 3467–3474 (2021). 40. Spica, R., Cristofalo, E., Wang, Z., Montijano, E. & Schwager, M. A real-time game theoretic planner for autonomous two-player drone racing. IEEE Trans. Robot. **36**, 1389–1403 (2020). 41. Day, B. L. & Fitzpatrick, R. C. The vestibular system. Curr. Biol. **15**, R583–R586 (2005). 42. Kim, J. et al. Esports arms race: Latency and refresh rate in competitive gaming tasks. J. Vis. **19**, 218c (2019).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate

Credit the original author and source, provide a link to the Creative Commons license, and indicate whether changes were made. Images or other third-party materials in this article are included in the article's Creative Commons license unless otherwise stated in the credit line for the material. If the material is not included in the article's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this license, visithttp://creativecommons.org/licenses/by/4.0/.

© The Author(s)
2023

方法

Quadrotor Simulation

四Rotor Dynamics

.To enable large-scale training, we used a high-fidelity simulation of quadrotor dynamics. This section briefly describes the simulation. The vehicle dynamics can be written as

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{p}}_{\mathcal{WB}} \\ \dot{\mathbf{q}}_{\mathcal{WB}} \\ \dot{\mathbf{v}}_{\mathcal{W}} \\ \dot{\boldsymbol{\omega}}_{\mathcal{B}} \\ \dot{\Omega} \end{bmatrix} = \begin{bmatrix} \mathbf{v}_{\mathcal{W}} \\ \mathbf{q}_{\mathcal{WB}} \cdot \begin{bmatrix} 0 \\ \boldsymbol{\omega}_{\mathcal{B}}/2 \end{bmatrix} \\ \frac{1}{m}(\mathbf{q}_{\mathcal{WB}} \odot (\mathbf{f}_{\text{prop}} + \mathbf{f}_{\text{aero}})) + \mathbf{g}_{\mathcal{W}} \\ J^{-1}(\boldsymbol{\tau}_{\text{prop}} + \boldsymbol{\tau}_{\text{mot}} + \boldsymbol{\tau}_{\text{aero}} + \boldsymbol{\tau}_{\text{iner}}) \\ \frac{1}{k_{\text{mot}}}(\Omega_{\text{ss}} - \Omega) \end{bmatrix}, \quad (1)$$

where $\odot$ represents the quaternion rotation, $\mathbf{p}$ $\mathbf{q}$ $\mathbf{v}_{\mathcal{WB}}$ $_{\mathcal{WB}}$ $_{\mathcal{W}}$, , and $\boldsymbol{\omega}_{\mathcal{B}}$ represent the quadrotor's position, attitude quaternion, inertial velocity, and body angular rate, respectively. The motor time constant is k_{mot}, and the motor speeds $\boldsymbol{\Omega}$ and $\boldsymbol{\Omega}_{\text{ss}}$ are the actual motor speed and the steady-state motor speed, respectively. The matrix J is the quadrotor's inertia, and $_{\mathcal{W}}\mathbf{g}$ represents the gravity vector. Two forces act on the quadrotor: the lift generated by the propellers $\mathbf{f}_{\text{prop}}$, and an aerodynamic force $\mathbf{f}_{\text{aero}}$, which aggregates all other forces such as aerodynamic drag, dynamic lift, and induced drag. The torque is modeled as the sum of four components: the torque generated by the thrust of each propeller $\boldsymbol{\tau}_{\text{prop}}$, the yaw torque generated by changes in motor speed $\boldsymbol{\tau}_{\text{mot}}$, the aerodynamic torque accounting for various aerodynamic effects such as blade flapping $\boldsymbol{\tau}_{\text{aero}}$, and the inertial term $\boldsymbol{\tau}_{\text{iner}}$. Each component is shown below

$$\mathbf{f}_{\text{prop}} = \sum_i \mathbf{f}_i, \ \boldsymbol{\tau}_{\text{prop}} = \sum_i \boldsymbol{\tau}_i + \mathbf{r}_{\text{P},i} \times \mathbf{f}_i, \quad (2)$$

$$\boldsymbol{\tau}_{\text{mot}} = J_{\text{m+p}} \sum_i \boldsymbol{\zeta}_i \dot{\Omega}_i, \ \boldsymbol{\tau}_{\text{iner}} = -\boldsymbol{\omega}_{\mathcal{B}} \times \mathbf{J}\boldsymbol{\omega}_{\mathcal{B}} \quad (3)$$

where $\mathbf{r}_{P,i}$ is the position of propeller i in the body frame, $\mathbf{f}_i$ and $\boldsymbol{\tau}_i$ are respectively the force and torque generated by the i-th propeller. The rotation axis of the i-th motor is denoted as $\boldsymbol{\zeta}_i$, the combined inertia of the motor and propeller is $J_{\text{m+p}}$, and the derivative of the rotational speed of the i-th motor is $\dot{\Omega}_i$. Each propeller is modeled using the commonly used quadratic model, which assumes that the lift force and drag torque are proportional to the square of the propeller rotational speedΩ_i:

$$\mathbf{f}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{l}} \cdot \Omega_i^2 \end{bmatrix}^\top, \quad \boldsymbol{\tau}_i(\Omega_i) = \begin{bmatrix} 0 & 0 & c_{\text{d}} \cdot \Omega_i^2 \end{bmatrix}^\top \quad (4)$$

where c_{l} and c_{d} denote the propeller lift coefficient and drag coefficient, respectively.

Aerodynamic forces and moments. Aerodynamic forces and moments are difficult to model using first-principles methods. Therefore, we use a data-driven model [43]. To maintain the low computational complexity required for large-scale reinforcement learning training, a gray-box polynomial model is adopted instead of a neural network. It is assumed that aerodynamic effects mainly depend on the velocity $\mathrm{v}_{\mathcal{B}}$ (in the body coordinate frame) and the square of the average motor speed Ω^2. The aerodynamic forces f_x, f_y, and f_z, as well as the moments τ_x, τ_y, and τ_z, are all esti mated in the body coordinate frame. The quantities v_x, v_y, and v_z represent the three axial velocity components (in the body coordinate frame), while v_{xy} represents the speed of the quadrotor in the (x, y) plane. Based on insights into the underlying physical process, linear and quadratic combinations of terms are selected. For readability, the coefficients multiplying each summation term have been omitted:

$$\begin{aligned}
f_x &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\ \overline{\Omega^2} \\
f_y &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\ \overline{\Omega^2} \\
f_z &\sim v_z + v_z|v_z| + v_{xy} + v_{xy}^2 + v_{xy}\ \overline{\Omega^2} + v_z\ \overline{\Omega^2} + v_{xy}\, v_z\ \overline{\Omega^2} \\
\tau_x &\sim v_y + v_y|v_y| + \overline{\Omega^2} + v_y\ \overline{\Omega^2} + v_y|v_y|\ \overline{\Omega^2} \\
\tau_y &\sim v_x + v_x|v_x| + \overline{\Omega^2} + v_x\ \overline{\Omega^2} + v_x|v_x|\ \overline{\Omega^2} \\
\tau_z &\sim v_x + v_y
\end{aligned}$$

The respective coefficients are then identified from real-world flight data, in which motion capture is used to provide ground-truth force and moment measurements. We use data from the track so that the dynamics model can fit that track. This is similar to how a human pilot trains on the specific track where the race will be held for days or weeks before the competition. In our case, the hu man pilot obtained one week of practice on the same track before the race.

Betaflight low-level controller. To control the quadrotor, the neural network outputs collective thrust and body angular rates. It is known that such control signals possess both high agility and good robustness for sim-to-real transfer[44]。 Subsequently, the predicted collective thrust and body angular rates are processed by the onboard low-level controller, which computes individual motor commands, and these commands are then converted into analog voltage signals through the electronic speed controllers (ESCs) that control the motors. On the physical aircraft, this low-level proportional-integral-derivative (PID) controller and the ESCs are implemented using the open-source Betaflight and BLHeli32 firmware[45]. In simulation, we use an accurate model of the low-level controller and the motor speed controller.

Because the Betaflight PID controller is optimized for manually piloted fli ght, it exhibits several peculiarities that the simulation correctly captures: the reference value of the D term is always zero (pure damping), the I term is reset when the throttle is cut, and when the motor thrust saturates, body angular rate control is given priority (all motor signals are scaled down proportionally to avoid saturation). The controller gains used for the simulation were identified from detailed logs of the Betaflight controller's internal state. The simulation is able to predict the individual motor commands with an error of less than 1%.

Battery model and electronic speed controllers. The low-level controller converts the individual motor commands into pulse-width modulation (PWM) signals and sends them t o the electronic speed controllers, which control the motors. Since the electronic speed controllers do not perform closed-loop control of the motor speed, the steady-state motor speed$\Omega_{i,\ \text{ss}}$ for a given PWM motor command cmd_i is a function of the battery voltage. Therefore, our simulation uses a gray-box battery model to model the battery voltage[46], which simulates the voltage based on the instantaneous power consumptionP_{mot}:

$$P_{\text{mot}} = \frac{c_{\text{d}}\Omega^3}{\eta} \quad (5)$$

Battery model[46] Then simulate the battery voltage according to this power demand. Given the battery voltageU_{bat} and each motor command$u_{\text{cmd},\ i}$, we use this mapping (again omitting the coefficients multiplied with each summation term)

$$\Omega_{i,\text{ss}} \sim 1 + U_{\text{bat}} + \sqrt{u_{\text{cmd},i}} + u_{\text{cmd},i} + U_{\text{bat}}\sqrt{u_{\text{cmd},i}} \quad (6)$$

to compute the corresponding steady-state motor speeds$\Omega_{i,\ \text{ss}}$ This is required for the dynamics simulation in equation (1). These coefficients have been

identified from Betaflight logs containing measurements of all relevant quantities. Combined with the low-level controller model, this enables the simulator to correctly convert actions expressed as collective thrust and body angular rates into desired motor speeds Ω_{ss}, see equation (1).

策略训练

We train a deep neural control policy that directly maps observations $\mathbf{o}_t$ in the form of platform state and next-gate observations to control actions $\mathbf{u}_t$, where actions are represented as mass-normalized collective thrust and body angular rates[44]. The control policy is trained in simulation using model-free reinforcement learning.

训练算法。训练使用近端策略优化[31]进行。这种演员-评论家方法需要在训练期间联合优化两个神经网络：策略网络，将观测映射到动作；以及价值网络，充当“评论家”并评估策略所采取的动作。训练后，仅策略网络部署在机器人上。

观测、动作和奖励。在时间t从环境中获得的观测$\mathbf{o}_t \in \mathbb{R}^{31}$包括：(1) 当前机器人状态的估计；(2) 赛道上需要通过的下一个门的相对位姿；以及 (3) 上一步施加的动作。具体地，机器人状态的估计包含平台的位置、其速度以及用旋转矩阵表示的态度，从而得到$\mathbb{R}^{15}$中的一个向量。尽管仿真内部使用四元数，我们使用旋转矩阵来表示态度以避免歧义[47]。下一个门的相对位姿通过提供四个门角相对于飞行器的相对位置来编码，从而得到$\mathbb{R}^{12}$中的一个向量。所有观测在传递给网络之前都被归一化。由于价值网络仅在训练时使用，它可以访问策略无法访问的环境特权信息[48]。此特权信息与策略网络的其他输入拼接，并包含机器人的精确位置、朝向和速度。

For each observation $\mathbf{o}_t$, the policy network produces an action $_t \in \mathbb{R}^4$ **a**, in the form of the desired mass-normalized collective thrust and body angular rates.

We use a dense shaping reward formulation to learn the perception-aware autonomous drone racing task. The reward r_t at time step t is given by

$$r_t = r_t^{\text{prog}} + r_t^{\text{perc}} + r_t^{\text{cmd}} - r_t^{\text{crash}} \tag{7}$$

where r^{prog} rewards progress toward the next gate[35], r^{perc} encodes perception awareness by adjusting the vehicle's attitude so that the camera's optical axis points toward the center of the next gate, r^{cmd} rewards smooth actions, and r^{crash} is a binary penalty that is activated only upon collision with a gate or when the platform leaves a predefined bounding box. If r^{crash} is triggered, the training episode ends.

Specifically, the reward terms are

$$\begin{aligned} r_t^{\text{prog}} &= \lambda_1 [d_{t-1}^{\text{Gate}} - d_t^{\text{Gate}}] \\ r_t^{\text{perc}} &= \lambda_2 \exp[\lambda_3 \cdot \delta_{\text{cam}}^4] \end{aligned} \tag{8}$$

$$r_t^{\text{cmd}} = \lambda_4 \mathbf{a}_t^{\omega} + \lambda_5 \|\mathbf{a}_t - \mathbf{a}_{t-1}\|^2 \tag{9}$$

$$r_t^{\text{crash}} = \begin{cases} 5.0, & \text{if } p_z < 0 \text{ or in collision with gate} \\ 0, & \text{otherwise} \end{cases}$$

where d_t^{Gate} represents the distance from the aircraft's center of mass to the center of the next gate at time step t, and δ_{cam} represents the angle between the camera's optical axis and the center of the next gate

while $_t$ $^{\omega}$**a** is the commanded body angular rate. The hyperparameters λ_1, ..., λ_5 are used to balance different terms (Extended Data Table 1a).

Training details. Data collection is performed by simulating 100 agents in parallel, which interact with the environment in episodes of 1,500 steps. At each environ ment reset, each agent is initialized at a random gate on the track, with a bounded perturbation around the state observed when it previously passed through that gate. Compared with prior work[44,49,50],we do not perform randomization of the platform dynamics during training. Instead, we fine-tune based on real-world data. The training environment is implemented using TensorFlow Agents[51]. Both the policy network and the value network are represented by two-layer perceptrons, each with 128 nodes, and use the LeakyReLU activation function with a negative slope of 0.2. The network parameters are optimized using the Adam optimizer, with a learning rate of 3×10^{-4}, for the policy network and the value network.

The policy was trained for a total of 1×10^8 environment interactions, which takes 50 minutes on a workstation (i9 12900K, RTX 3090, 32 GB RAM DDR5). Fine-tuning, on the other hand, involves 2×10^7 environment interactions.

Residual Model Identification

We fine-tune the original policy based on a small amount of data collected in the real world. Specifically, we collect three complete rollouts in the real world, corresponding to approximately 50 seconds of flight time. We fine-tune the policy by identifying residual observations and residual dynamics, which are then used for simulation training. In this fine-tuning stage, only the weights of the control policy are updated, while the weights of the gate detection network remain unchanged.

Residual Observation Model. High-speed flight causes significant motion blur, which can lead to the loss of tracked visual features and severe drift in linear odometry estimates. We fine-tune the policy using an odometry model identified from only a small number of trials recorded in the real world. To model odometry drift, we use Gaussian processes[36], , as they allow fitting the posterior distribution of odometry disturbances, from which we can sample temporally consistent realizations.

Specifically, the Gaussian process model fits the residual position, velocity, a nd attitude as functions of the ground-truth robot state. The observation residuals are identified by comparing the visual-inertial odometry (VIO) estimates observed during real-world rollouts with the ground-truth platform states obtained from an external motion tracking system.

We treat each dimension of the observation separately, thereby effectively fitting a set of nine one-dimensional Gaussian processes to the observation residuals. We use a mixture of radial basis function kernels

$$\kappa(\mathbf{z}_i, \mathbf{z}_j) = \sigma_{\text{f}}^2 \exp\left(-\frac{1}{2}(\mathbf{z}_i - \mathbf{z}_j)^{\top} L^{-2} (\mathbf{z}_i - \mathbf{z}_j)\right) + \sigma_{\text{n}}^2 \tag{10}$$

where L is a diagonal length-scale matrix, σ_{f} and σ_{n} denote the data and prior noise variances, respectively, and $\mathbf{z}_i$ and $\mathbf{z}_j$ denote data features. The kernel hyperparameters are optimized by maximizing the log marginal likelihood. After kernel hyperparameter optimization, we sample new realizations from the posterior distribution, which are then used for fine-tuning the policy. Extended Data Fig. 1 shows the residual observations of position, velocity, and attitude during real-world rollouts, as well as 100 realizations sampled from the Gaussian process model.

Residual dynamics model. We use a residual model to supplement the simulated robot dynamics[52]. Specifically, we identify the residual acceleration as a function of the platform state**s**and the command mass-normalized collective thrust$c$:

$$\mathbf{a}_{\mathbf{res}} = \mathrm{KNN}(\mathbf{s}, \mathbf{c}) \tag{11}$$

We use k-nearest neighbor regression, where $k = 5$. The size of the dataset used for residual dynamics model identification depends on the track layout, and for the track layout used in this work, it ranges between 800 and 1,000 samples.

门检测

To correct the accumulated drift of the VIO pipeline, the gate is used as a distinctive landmark for relative localization. Specifically, the gate is detected by segmenting the gate corners in the onboard camera view[26]. The grayscale image provided by the Intel RealSense tracking camera T265 is used as the i nput image to the gate detector. The architecture of the segmentation network is a six-level U-Net[53], with each level having (8, 16, 16, 16, 16, 16) convolutional filters of sizes (3, 3, 3, 5, 7, 7), and a final additional layer operating on the U-Net output containing 12 filters. The activation function uses LeakyReLU,$\alpha = 0.01$. For deployment on the NVIDIA Jetson TX2, the network was ported to TensorRT. To optimize memory usage and computation time, inference is performed in half-precision mode (FP16), and the image is downsampled to384×384 in size before being input to the network. On the NVIDIA Jetson TX2, one forward pass takes 40 ms.

VIO Drift Estimation

The odometry estimates from the VIO pipeline[54] exhibit significant drift during high-speed flight. We use gate detection to stabilize the pose estimates produced by VIO. The gate detector outputs the corner coordinates of all visible gates. First, the relative pose of all predicted gate estimates is obtained using infinitesimal plane-based pose estimation (IPPE)[34]. Given this relative pose estimate, each gate observation is assigned to the nearest gate in the known track layout, thereby yielding the drone's pose estimate.

Since the gate detection frequency is low and the VIO orientation esti mates are of high quality, we only correct the translation component of the VIO measurements. We use a Kalman filter to estimate and correct the drift of the VIO pipeline; this filter estimates the translational drift$\mathbf{p}_{\mathrm{d}}$ (position offset) and its derivative, i.e., the drift velocity$\mathbf{v}_{\mathrm{d}}$. Drift correction is performed by subtracting the estimated drift states$\mathbf{p}_{\mathrm{d}}$ and$\mathbf{v}_{\mathrm{d}}$ from the corresponding VIO estimates. The Kalman filter state$\mathbf{x}$ is given by$\mathbb{R}\mathbf{x}\ \mathbf{p}\ \mathbf{v} = [\ _{\mathrm{d}}^{\top},\ \ ^{\top}]^{\top} \in_{\mathrm{d}}{}^{6}$.

The update of state $\mathbf{x}$ and covariance P is given by:

$$\mathbf{x}_{k+1} = F\mathbf{x}_k, \quad P_{k+1} = FP_kF^{\top} + Q, \tag{12}$$

$$F = \begin{bmatrix} \mathbb{I}^{3\times3} & \mathrm{d}t\,\mathbb{I}^{3\times3} \\ 0^{3\times3} & \mathbb{I}^{3\times3} \end{bmatrix}, \quad Q = \begin{bmatrix} \sigma_{\mathrm{pos}}\mathbb{I}^{3\times3} & 0^{3\times3} \\ 0^{3\times3} & \sigma_{\mathrm{vel}}\mathbb{I}^{3\times3} \end{bmatrix}. \tag{13}$$

Based on the measurement results, the process noise is set to $\sigma_{\mathrm{pos}} = 0.05$ and $\sigma_{\mathrm{vel}} = 0.1$. The filter state and covariance are initialized to zero. For each measurement $\mathbf{z}_k$ (the pose estimate from gate detection), the predicted VIO drift $\mathbf{x}_k^-$ is corrected to the estimate $\mathbf{x}_k^+$ according to the Kalman filter equations:

$$\begin{aligned} K_k &= P_k^- H_k^{\top}(H_k P_k^- H_k^{\top} + R)^{-1}, \\ \mathbf{x}_k^+ &= \mathbf{x}_k^- + K_k(\mathbf{z}_k - H(\mathbf{x}_k^-)), \\ P_k^+ &= (I - K_k H_k)P_k^-, \end{aligned} \tag{14}$$

where K_k is the Kalman gain, R is the measurement covariance, and H_k is the measurement matrix. If multiple gates are detected in a single camera frame, all relative pose estimates are stacked and processed in the same Kalman filter update step. The main source of measurement error is the uncertainty in the network's gate corner detection. This error in the image plane leads to pose error when IPPE is applied.

We chose a sampling-based method to estimate the pose error based on the known average gate corner detection uncertainty. For each gate, the IPPE algorithm is applied both to the nominal gate observation and to 20 perturbed gate corner estimates. The resulting distribution of pose estimates is then used to approximate the measurement covariance R of the gate observation.

仿真结果

在自主无人机竞速中达到冠军级表现需要克服两个挑战:不完美感知和系统动力学的不完整模型。在仿真中的受控实验里,我们评估我们的方法对这两个挑战的鲁棒性。为此,我们评估在四种不同设置下部署时竞速任务中的性能。在设置(1)中,我们仿真一个简化四旋翼模型,并访问真值状态观测。在设置(2)中,我们用从真实世界飞行中识别出的噪声观测替换真值状态观测。这些噪声观测通过从残差观测模型中采样一个实现来生成,并且与所部署控制器的感知意识无关。设置(3)和(4)分别与前两个设置共享观测模型,但将简化动力学模型替换为更准确的空气动力学仿真[43]。这四种设置允许对方法对动力学变化和观测保真度变化的敏感性进行受控评估。

在所有四种设置中，我们将我们的方法与以下基线进行基准比较：零样本、域随机化和时间最优。零样本基线代表一个基于学习的竞速策略[35]，该策略使用无模型强化学习训练，并从训练域零样本部署到测试域。该策略的训练域等于实验设置(1)，即理想化动力学和真值观测。域随机化通过随机化观测和动力学属性来扩展零样本基线的学习策略，以提高鲁棒性。时间最优基线使用预计算的时间最优轨迹[28]，并通过MPC控制器进行跟踪。与其他基于模型的时间最优飞行方法相比，该方法已展现出最佳性能[55,56]。轨迹生成和MPC控制器所使用的动力学模型与实验设置(1)中的模拟动力学相匹配。

Performance evaluation is based on the fastest lap time, the average and minimum observed gate margin for successful gate passages, and the percentage of successfully completed track. The gate margin measures the distance between the UAV and the nearest point on the gate when it crosses the gate plane. A larger gate margin indicates that the quadrotor passes near the center of the gate. Preserving a smaller gate margin can improve speed, but it also increases the risk of collision or missing the gate. Any lap that results in a crash is considered invalid.

The results are summarized in Extended Data Table 1c. When deployed with idealized dynamics and ground-truth observations, all methods successfully complete the task, with the time-optimal baseline achieving the lowest lap time. When deployed in settings with domain shift, whether in dynamics or observations, the performance of all baselines collapses, and none of the three baselines can complete even a single lap. This performance degradation is observed in both learning-based methods and traditional methods. In contrast, our method, characterized by an empirical model of the dynamics and observation noise, succeeds in all deployment settings, with only a small increase in lap time.

A key feature that enables our method to succeed under various deployment conditions is the use of empirical models of dynamics and observation noise estimated from real-world data. A comparison between methods that have access to such data and those that do not is not entirely fair. Therefore, we also benchmark the performance of all baseline methods when they have access to the same real-world data used by our method. Specifically, we

compare performance in experimental setup (2), which has an idealized dynamics model but noisy perception. All baseline methods are provided with predictions from the same Gaussian process model that we use to characterize observation noise. The results are summarized in Extended Data Table 1b. All baseline methods benefit from more realistic observations, thereby achieving higher completion rates. However, our method is the only one that can reliably complete the entire track. In addition to predictions from the observation noise model, our method also accounts for model uncertainty. For an in-depth comparison of reinforcement learning and optimal control performance in controlled experiments, we refer the reader to reference57.

Fine-tuning with multiple iterations

We investigated the degree of behavioral change across different iterations. Our analysis results show that subsequent fine-tuning operations have a negligible impact on performance improvement and behavioral change (Extended Data Fig. 2).

Below, we provide more details about this investigation. We first enumerate the fine-tuning steps to provide the necessary notation:

1. Train policy-0 in simulation. 2. Deploy policy-0 in the real world. This policy operates based on ground-truth data from the motion capture system. 3. Identify the residuals observed by policy-0 in the real world. 4. Based on the identified residuals, fine-tune policy-0 to train policy-1. 5. Deploy policy-1 in t he real world. This policy operates solely based on onboard sensor measurements. 6. Identify the residuals observed by policy-1 in the real world. 7. Based on the identified residuals, fine-tune policy-1 to train policy-2.

After fine-tuning on the respective residuals, we compared the performance of Policy-1 and Policy-2 in simulation. The results are shown in Extended Data Fig. 2. We observed that the difference in distance from the gate center (an indicator of policy safety) was 0.09 ± 0.08 m. In addition, the difference in time required to complete a single lap was 0.02 ± 0.02 s. Note that this lap-time difference is much smaller than the difference between Swift and the human pilot in single-lap completion time (0.16 s).

Drone Hardware Configuration

人类飞行员和Swift所使用的四旋翼飞行器具有相同的重量、形状和推进系统。该平台设计基于Agilicious框架[58]。每个飞行器的重量为870 g，可产生约35 N的最大静态推力，从而得到4.1的静态推重比。每个平台的底座由一个Armattan Chameleon 6英寸主机架构成，该主机架配备T-Motor Velox 2306电机和5英寸三叶螺旋桨。自主无人机的主要计算资源由NVIDIA Jetson TX2及Connect Tech Quasar载板提供，其具有一个运行频率为2 GHz的六核CPU和一个运行频率为1.3 GHz、拥有256个CUDA核心的专用GPU。尽管门检测网络的前向传播在GPU上执行，竞速策略在CPU上评估，一次推理过程耗时8 ms。自主无人机搭载一个Intel RealSense追踪相机T265，其以100 Hz提供VIO估计[59]，并通过USB馈送至NVIDIA Jetson TX2。人类驾驶的无人机既不搭载Jetson计算机也不搭载RealSense相机，而是配备相应的配重。由人类飞行员或Swift生成的形式为集体推力和机体角速率的控制指令被发送至商用飞行控制器，该控制器运行在216 MHz的STM32处理器上。飞行控制器运行Betaflight，这是一款开源飞行控制软件[45]。

Impressions of the Human Pilot

The following quotes convey the impressions of three human champions who raced against Swift.

Alex Vanover:

• These races will be decided at the half-roll inverted maneuver, which is the most challenging part of the track.

• This is the best race! I was very close to the autonomous drone, and while trying to keep up with it, I could really feel the turbulence.**Th**

omas Bitmatta:

• The possibilities are infinite, and this is the beginning of something that could change t he entire world. On the other hand, I am a racer, and I do not want anything to be faster than me.

• The faster you fly, the more you have to trade precision for speed. • It is inspiring to see the true potential of drones. Soon, AI drones may even be used as training tools to understand what will become possible.

Marvin Schaepper:

• Racing against a machine feels different because you know the machine will not get tired.

Research Ethics

This study was conducted in accordance with the Declaration of Helsinki. According to the regulations of the University of Zurich, this study protocol was exempt from review by the ethics committee because no health-related data were collected. All participants provided written informed consent before participating in the study.

Data Availability

All (other) data required to evaluate the conclusions of the paper are included in the paper or the extended data. The motion capture recordings of the competition events and the accompanying analysis code can be found in the file 'racing_ data.zip' on Zenodo at https://doi.org/10.5281/zenodo.7955278.

Code Availability

Swift pseudocode detailing the training process and algorithms can be found i n the file 'pseudocode.zip' on Zenodo athttps://doi.org/10.5281/zenodo.7955278. To prevent potential misuse, the complete source code related to this study will not be made publicly available.

43. Bauersfeld, L., Kaufmann, E., Foehn, P., Sun, S. & Scaramuzza, D. in *Proc. Robotics:Science and SystemsXVII* 42 (Robotics: Science and Systems Foundation, 2021).44. Kaufmann, E., Bauersfeld, L. & Scaramuzza, D. in*Proc. 2022 International Conference onRobotics and Automation (ICRA)* 10504–10510 (IEEE, 2022).45. TheBetaflight Open Source Flight Controller Firmware Project. Betaflight. https://github.com/betaflight/betaflight(2022).46. Bauersfeld, L. & Scaramuzza, D. Range, endurance, and optimal speed estimates formulticopters.*IEEE Robot. Autom. Lett.* **7**, 2953–2960 (2022).47. Zhou, Y., Barnes, C., Lu, J., Yang, J. & Li, H. in *Proc. IEEE/CVF*Computer Vision and Pattern Recognition Conference (*CVPR*) 5745–5753 (IEEE, 2019).48. Pinto, L., Andrychowicz, M., Welinder, P.,Zaremba, W. & Abbeel, P. in *Proc. Robotics:Science and Systems XIV* (MIT Press Journals, 2018).49.Molchanov, A. et al. in *Proc. 2019 IEEE/RSJ International Conference on Intelligent Robotsand Systems (IROS)* 59–66 (IEEE, 2019).50. Andrychowicz, O. M. et al. Learning dexterous in-hand manipulation. *Int. J. Robot. Res.***39**,3–20 (2020).51. Guadarrama, S. et al. TF-Agents: a library for reinforcement learning inTensorFlow.https://github.com/tensorflow/agents (2018).52. Torrente, G., Kaufmann, E., Foehn, P. &Scaramuzza, D. Data-driven MPC for quadrotors.*IEEE Robot. Autom. Lett.* **6**, 3769–3776 (2021).53.Ronneberger, O., Fischer, P. & Brox, T. in *Proc.* International Conference on Medical Image Computing and Computer-Assisted Intervention 234–241 (Springer, 2015).54. Intel RealSense T265 series product fa mily.h

t
t
p
s://
w
w
w.
i
n
t
e
lr
ealsense.com/wp-content/uploads/2019/09/Intel_RealSense_Tracking_Camera_Datasheet_Rev004_release.pdf (2019).55. Ryou, G., Tal, E. & Karaman, S. Multi-fidelity black-box optimization for time-optimalquadrotor maneuvers. *Int. J. Robot. Res.* **40**, 1352–1369 (2021).56. Pham, H. & Pham, Q.-C. A new approach to time-optimal path parameterization based onreachability analysis. *IEEE Trans. Robot.* **34**, 645–659 (2018).57. Song, Y., Romero, A., Müller, M., Koltun, V. & Scaramuzza, D. Reaching the limit inautonomous racing: optimal control versus reinforcement learning. *Sci. Robot.* (in thepress).

58. Foehn, P. et al. Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight. *Sci. Robot.***7**, eabl6259 (2022). 59. Jones, E. S. & Soatto, S. Visual-inertial navigation, mapping and localization: A scalable real-time causal approach. *International Journal of Robotics Research* **30**, 407–430 (2011).

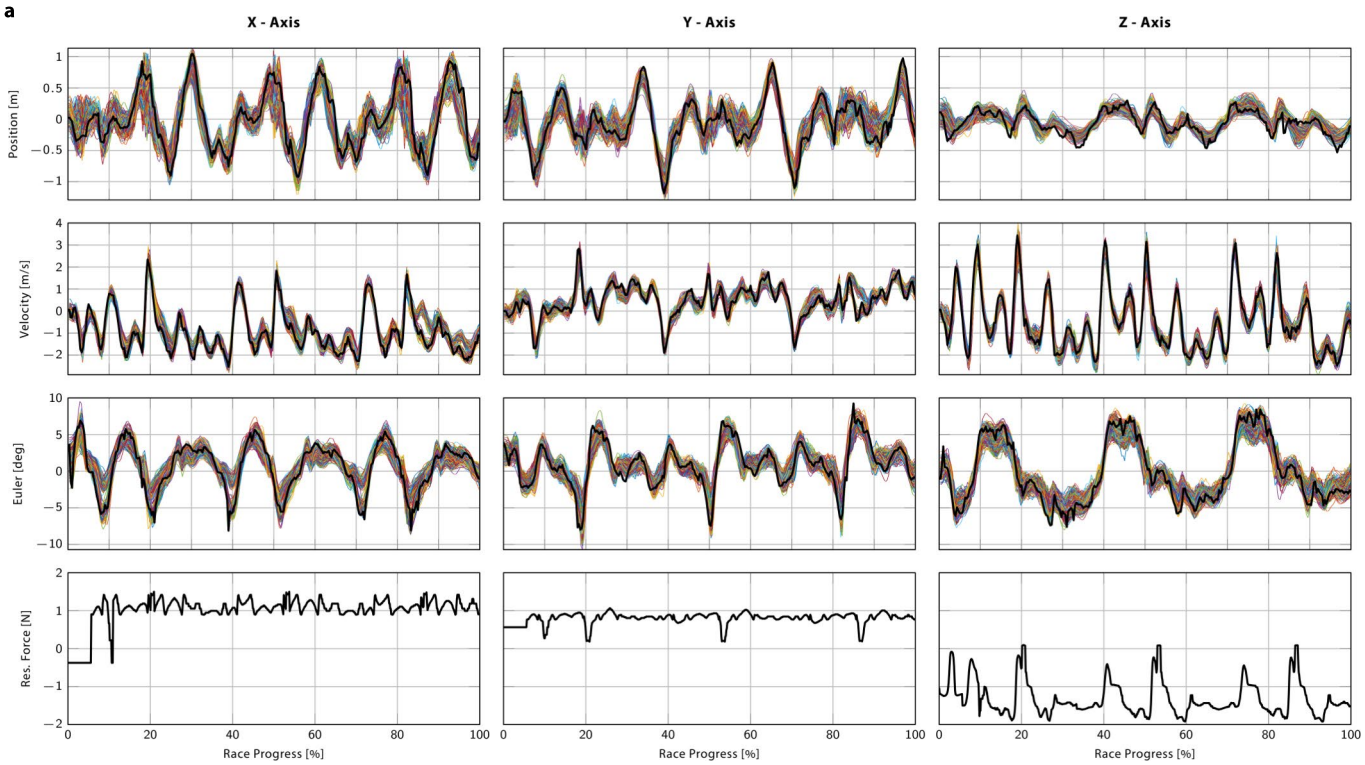
Acknowledgments The authors thank A. Vanover, T. Bitmatta and M. Schaepper for accepting to race against Swift. The authors also thank C. Pfeiffer, T. Längle and A. Barden for their contributions to the event organization and drone hardware design. This work was supported by the Intel Embodied Intelligence Lab, the Swiss National Science Foundation (SNSF) through the National Centre of Competence in Research (NCCR) Robotics, and the European Research Council (ERC) under grant agreement 864042 (AGILEFLIGHT).

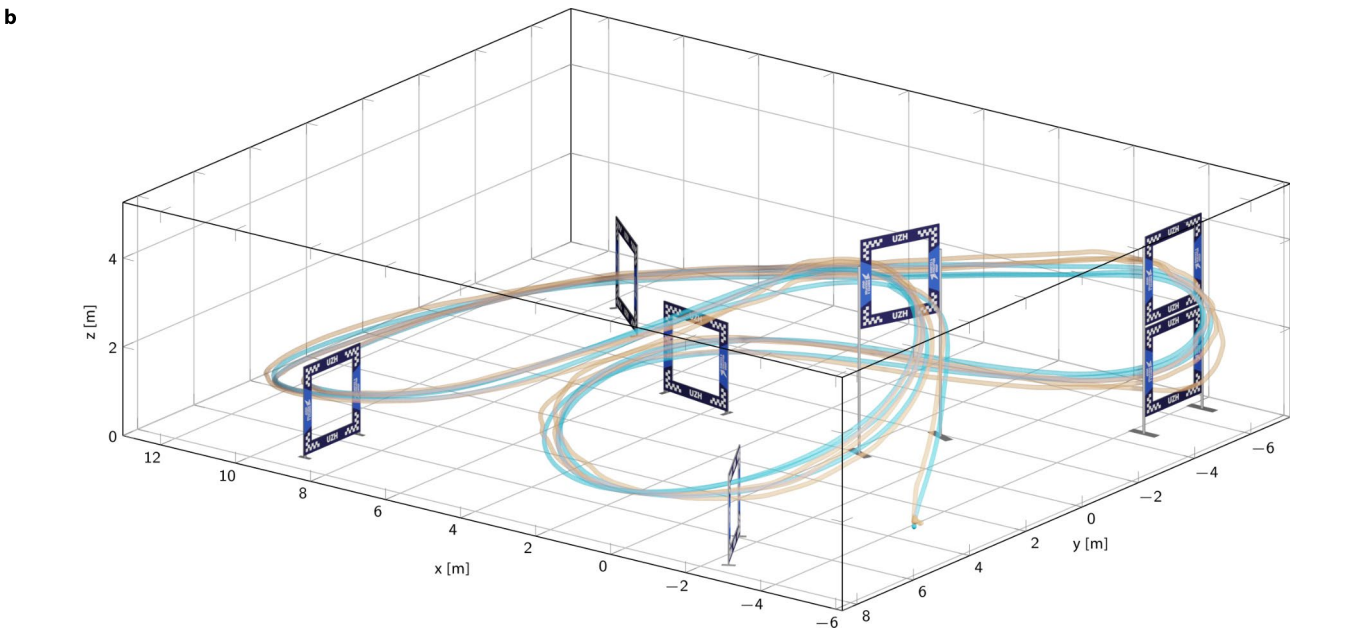
Author Contributions E.K. proposed the core idea, implemented the system, completed the experiments and data analysis, and wrote the paper. L.B. contributed to the core idea, experiments, data analysis, and paper writing, and designed the graphical illustrations. A.L. proposed the core idea, and contributed to the experimental design, data analysis, and paper writing. M.M. contributed to the experimental design, data analysis, and paper writing. V.K. contributed to the core idea, experimental design, experimental analysis, and paper writing. D.S. contributed to the core idea, experimental design, experimental analysis, and paper writing, and provided funding.

Conflicts of Interest The authors declare no conflicts of interest.

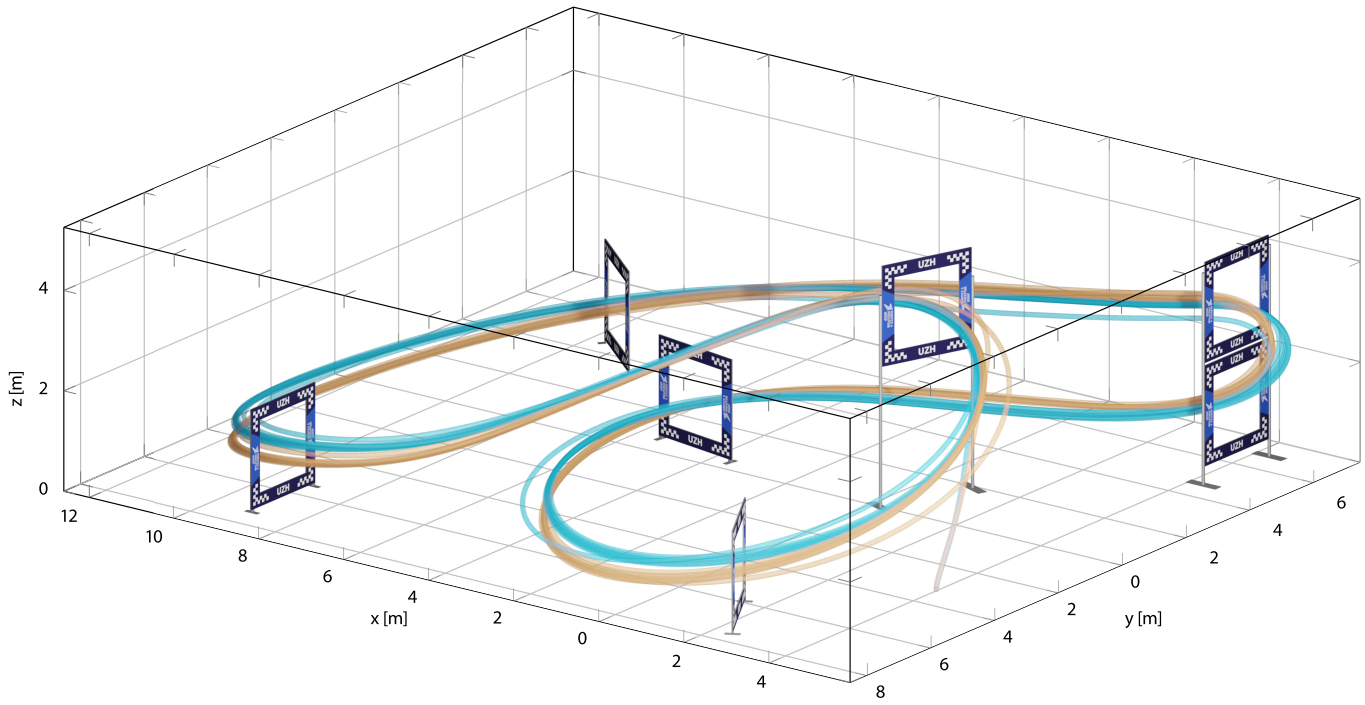
Additional Information

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1038/s41586-023-06419-4.**Corresponding author and material requests**Please contact Elia Kaufmann.**Peer review information** *Nature* Thanks to Sunggoo Jung, Karime Pereida and other anonymous reviewers for their contributions to the peer review of this work. Peer review reports are available.**Reprints and permission information** Available athttp://www.nature.com/reprints.





Extended Data Figure 1 | Residual model. a, Visualization of the residual observation model and residual dynamics model identified from real-world data. The black curve depicts the residuals observed in the real world, and the colored lines show 100 sampled re alizations of the residual observation model. Each subplot depicts the entire race, i.e., three laps.**b**, Observations of the prediction residuals of the simulated deployment. Blue, the ground-truth positions provided by the simulator; orange, the perturbed positions generated by the Gaussian process residuals.



Extended Data Figure 2 | Multi-iteration fine-tuning. Comparison of rollouts after fine-tuning the policy for one iteration (blue) and two iterations (orange)

e).

Extended Data Table 1| Network parameters and detailed metrics comparing our Swift system with other methods

a

Hyperparameter	Value
γ (discount factor)	0.99
ε (importance ratio clipping)	0.2
λ_1	1.0
λ_2	0.02
λ_3	-10.0
λ_4	-2e-4
λ_5	-1e-4

b

Approach	Lap time in seconds	Gate margin in meters	Completion in %
Zero-Shot Transfer [33]	4.92	0.57 — 0.41	42
Domain Randomization	1	0.34 — 0.23	19
Time-Opt. Traj. + MPC [26]	1	0.51 — 0.41	19
Ours	5.26	0.56 — 0.44	100

c

		Ground-truth observations			Noisy observations		
	Approach	Lap time in seconds	Gate margin in meters	Completion in %	Lap time in seconds	Gate margin in meters	Completion in %
Idealized dyn.	Zero-Shot Transfer [33]	4.88	0.63 — 0.46	100	1	n/a — n/a	0
	Domain Randomization	5.06	0.60 — 0.47	100	1	0.43 — 0.30	9
	Time-Opt. Traj. + MPC [26]	4.60	0.50 — 0.25	100	1	0.48 — 0.29	9
	Ours	4.88	0.63 — 0.46	100	5.26	0.56 — 0.44	100
Realistic dyn.	Zero-Shot Transfer [33]	1	0.62 — 0.62	4	1	0.41 — 0.21	9
	Domain Randomization	1	0.28 — 0.28	4	1	0.47 — 0.45	9
	Time-Opt. Traj. + MPC [26]	1	n/a — n/a	0	1	0.23 — 0.23	4
	Ours	5.20	0.51 — 0.30	100	5.42	0.48 — 0.23	100

d

	Autonomous Drone					**Vanover**					**Bitmatta**					**Schaepper**				
	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m	Speed in m/s	Power in W	Thrust in N	Time in s	Length in m
Segment 1	**6.52**	**548.14**	**19.80**	**1.27**	8.27	5.62	348.91	14.96	1.42	7.98	5.93	365.78	15.87	1.36	8.07	4.92	201.06	10.47	1.59	**7.82**
Segment 2	13.72	852.20	29.01	**1.82**	**25.01**	**13.83**	**863.85**	**29.22**	1.83	25.31	13.72	854.36	28.95	1.89	25.94	10.76	508.29	19.97	2.36	25.35
Segment 3	13.70	798.69	27.60	**1.96**	**26.89**	**13.86**	**853.69**	**28.98**	2.09	28.91	13.16	727.36	25.50	2.23	29.35	12.66	636.78	23.47	2.40	30.34
Segment 4	13.43	**1047.05**	**33.69**	1.61	21.62	12.97	953.47	31.62	**1.60**	**20.72**	**13.47**	1035.91	33.39	1.67	22.53	13.11	885.53	29.95	1.77	23.20
Full Race	**13.11**	**866.48**	**29.16**	**17.46**	**228.85**	12.96	843.50	28.65	17.96	232.79	12.89	822.50	27.96	18.74	241.52	11.55	623.53	22.95	21.16	244.49

a, training hyperparameters.**b**, comparison with baselines that use the same observation noise model as our method.**c**, simulation evaluation, comparing idealized dynamics (top) with realistic dynamics (bottom) and ground-truth observations (left) with noisy observations (right). We report the fastest collision-free single-lap time (seconds), the average and minimum gate margin for successfully passing gates, and the track completion percentage. We compare our method with a learning-based method performing zero-shot transfer, with and without domain randomization during training, and with a traditional planning and control method[28].**d**, comparison of average speed, power, thrust, time, and flight distance for each pilot in the fastest flight race. Best values are shown in bold.